EXPLORING THE DATABASE MESH CONCEPT FOR DISTRIBUTED STATE MANAGEMENT IN MICROSERVICES ARCHITECTURES

Simbarashe Denzel Chingodza | EDUV4940179.

Department of Software Engineering, Faculty of Information Technology,
Eduvos, Mowbray, Cape Town, 7700, South Africa

Date of Submission: 8 June 2026

Abstract

The rapid proliferation of microservices architectures has introduced significant challenges in managing distributed state across independently deployed services. This systematic literature review explores the emerging concept of the database mesh an architectural paradigm that extends data mesh principles to operational, transactional databases within event-driven microservices environments. Drawing on nine peer-reviewed studies published between 2021 and 2025, this review investigates the principles, benefits, and architectural implications of the database mesh approach. Findings reveal that database mesh addresses critical limitations of polyglot persistence and distributed transaction management by promoting domain-oriented data ownership, self-serve data infrastructure, and federated governance. Key challenges including eventual consistency, saga-pattern complexity, and cross-service data integrity are examined. The review concludes with a conceptual architecture and actionable recommendations for researchers and practitioners adopting this paradigm.

Keywords: *database mesh, microservices, distributed state management, event driven architecture, polyglot persistence*

1. Introduction

1.1 Background

The adoption of microservices architecture has fundamentally transformed how modern software systems are designed, deployed, and scaled. By decomposing monolithic applications into fine-grained, independently deployable services, organisations gain flexibility, resilience, and continuous delivery capabilities. However, this architectural shift introduces a new class of complexity: the management of distributed state. Unlike monolithic systems where a single, shared database provides a unified view of application state, microservices architectures mandate that each service owns its own data store a principle known as database-per-service. This fragmentation, while promoting loose coupling, creates formidable challenges around data consistency, cross-service transactions, and observability (Laigner et al., 2021).

The concept of the database mesh emerges from a broader movement in data architecture. Dehghani's (2022) seminal work on the data mesh introduced the idea of treating data as a product owned by domain teams, governed through a federated model, and accessed via self-serve infrastructure. While originally conceived for analytical data platforms, the principles have begun to migrate toward operational microservices databases, giving rise to the notion of a database mesh: a distributed data governance and access layer that sits across microservice databases, providing unified policy enforcement, observability, and inter-service data coordination without sacrificing autonomy.

1.2 Problem Statement

Despite growing interest in microservices, most research and tooling have focused on service-to-service communication through service meshes (e.g., Istio, Linkerd), while the persistent data layer remains fragmented, inconsistent, and difficult to govern. Distributed transactions across polyglot data stores continue to be a leading source of data anomalies, outages, and developer complexity. The saga pattern, event sourcing, and CQRS each offer partial remedies, but no overarching framework exists for governing the distributed data plane in microservices the way a service mesh governs the network plane. The database mesh concept aims to fill this gap, yet it remains underexplored in peer-reviewed literature.

1.3 Aim of the Review

The aim of this systematic literature review is to explore the Database Mesh Concept for Distributed State Management in Microservices Architectures, synthesising current academic knowledge on its principles, benefits, challenges, and architectural design.

1.4 Research Questions

The central research questions this review seeks to answer is: *"How suitable is the database mesh concept for managing distributed state in event driven microservices?"* This question is pursued through the following objectives:

1. To investigate the concept and principles of a database mesh in distributed systems.
2. To identify potential benefits and challenges of adopting a database mesh approach.
3. To design a conceptual or prototype architecture incorporating a database mesh.

1.5 Significance of the Study

This review is significant for several groups. Software engineers and architects designing cloud-native systems will benefit from a consolidated understanding of distributed state management patterns and how the database mesh concept can reduce operational complexity. Technology organisations undergoing digital transformation will gain insight into governance models that can be applied to their microservices data layers. Academic researchers in the fields of distributed systems and software architecture will find a synthesis of current literature that highlights open research gaps. Finally, educators and curriculum designers in computing disciplines will find this review a useful resource for contextualising emerging data architecture paradigms within established microservices theory.

1.6 Scope of the Review

This review covers peer-reviewed journal articles, conference papers, and academic theses published between 2021 and 2025. Studies were sourced from ProQuest, Google Scholar, Scopus, and the Web of Science. The search was bounded to the topics of database mesh, distributed state management, event-driven microservices, polyglot persistence, saga patterns, CQRS, and event sourcing. Non-English publications, duplicate studies, and conference abstracts without full-text availability were excluded. The review does not cover service mesh tools (e.g., Istio, Linkerd) except where they intersect with database-layer concerns.

2. Methodology

2.1 Review Design

This study follows a Systematic Literature Review approach, adhering to the PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) framework. The SLR methodology was selected for its structured, replicable process that minimises selection bias and ensures transparent reporting of included and excluded literature. The positivist research paradigm underpins this review, treating the selected literature as an objective body of evidence from which generalisable insights on database mesh architecture can be inductively drawn.

2.2 Search Strategy

Academic databases searched included Google Scholar, ProQuest, Scopus, and the Web of Science. The following Boolean search strings were employed:

> *("database mesh" OR "data mesh") AND ("distributed state" OR "microservices") AND ("event-driven" OR "distributed transactions")*
>
> *("saga pattern" OR "CQRS" OR "event sourcing") AND ("microservices") AND (2021:2025)*
>
> *abstract(database mesh) AND abstract(distributed state) OR (event-driven microservices)*

The initial search returned approximately 3,484 results. After applying the date filter (2021–2025), 1,975 results remained. Limiting to peer-reviewed sources yielded 474 results, of which 296 were discipline-specific and formed the basis for further screening.

2.3 Inclusion Criteria

- Peer-reviewed journal articles and conference papers — 474
- Published between 2021 and 2025 — 474
- Written in the English language — 466
- Discipline-specific (distributed systems, software architecture, cloud computing) — 296
- Full-text availability confirmed (downloaded copy obtained) — 46 full texts assessed

2.4 Exclusion Criteria

- Duplicate studies
- Conference abstracts without full-text availability
- Non-English language publications
- Studies unrelated to distributed data management or microservices architecture — 250 excluded after discipline screening
- Grey literature without peer-review validation

2.5 Study Selection Process

Identification: Articles were identified through keyword searches across the four databases listed above. A total of 296 discipline-specific records were retrieved after applying date and language filters.

Screening: Titles and abstracts of all 296 records were screened for relevance to distributed state management, microservices data architecture, or database/data mesh concepts. Articles clearly unrelated to the topic were excluded at this stage, reducing the pool to 46 full-text articles assessed for eligibility. Screening focused on the abstract, introduction, and results sections of each article to determine topical relevance.

Eligibility: Full texts of 46 articles were reviewed against the three research objectives. Articles were assessed for conceptual alignment: (i) addressing the principles of database mesh or data mesh, (ii) examining benefits and challenges of distributed data management, and (iii) proposing or evaluating architectural patterns for distributed state. Studies satisfying at least one objective with substantive depth were retained.

Inclusion: Nine (9) articles were selected for final inclusion. These are presented in the table below

Included Studies

S/N	Author(s)	Title	Year	Journal/Conference
1	Laigner et al.	Data Management in Microservices: State of the Practice, Challenges, and Research Directions	2021	Proceedings of the VLDB Endowment (ACM)
2	Correia et al.	Data Mesh: A Systematic Gray Literature Review	2023	arXiv / IEEE Access
3	Nguyen et al.	Current Trends in the Management of Distributed Transactions in Micro-Services Architectures: A Systematic Literature Review	2023	Journal of Computer and Communications (SCIRP)
4	Pereira et al.	Enhancing Saga Pattern for Distributed Transactions within a Microservices Architecture	2022	Applied Sciences – MDPI
5	Rajasekaran et al.	A Survey of Saga Frameworks for Distributed Transactions in Event-Driven Microservices	2022	3rd International Conference on Smart Technologies in

S/N	Author(s)	Title	Year	Journal/Conference
				Computing, Electrical and Electronics (ICSTCEE)
6	Okonkwo & Hassan	Polyglot Persistence in Microservices: Managing Data Diversity in Distributed Systems	2025	IEEE Xplore
7	Anand et al.	Ensuring Data Consistency in Enterprise Systems Through Event-Driven Microservices and Distributed Transaction Management	2023	International Journal of Advanced Computer Science and Applications
8	Mehta & Vora	Dynamic Data Mesh and Data Fabric Approaches for Scalable Microservices Integration	2025	Journal of Emerging Technologies in Computing
9	Patel et al.	Exploring Event-Driven Architecture in Microservices: Patterns, Pitfalls and Best Practices	2025	International Journal of Software Engineering Research and Practices

3. Results / Findings

The following section presents findings from the nine selected studies, organised according to the three research objectives of this review. Each objective is explored through a description of major themes, a comparison across studies, and identification of key insights.

3.1 Objective 1: Concept and Principles of a Database Mesh in Distributed Systems

3.1.1 Main Themes

The database mesh concept extends the four foundational principles of the data mesh domain ownership, data as a product, self-serve data infrastructure, and federated computational governance into the operational data tier of microservices. Correia et al. (2023) identify these four pillars as the architectural backbone that distinguishes data mesh from earlier data warehouse or data lake paradigms. Their systematic grey literature review confirms that domain-oriented data ownership is the most frequently cited principle, appearing in over 80% of reviewed sources, making it the conceptual centrepiece of the database mesh paradigm.

Laigner et al. (2021), in their comprehensive study of 300 peer-reviewed articles and nine open-source microservice applications, establish that the dominant data management challenge in microservices is not querying or storage per se, but the coordination of state across independently owned databases. Their findings reveal that eventual consistency is adopted as the de facto consistency model by most practitioners, yet this choice introduces significant complexity in reasoning about distributed invariants a challenge the database mesh's federated governance layer aims to mitigate by standardising how services expose, consume, and govern shared state.

Mehta & Vora (2025) extend this further by examining dynamic data mesh and data fabric approaches, finding that a data mesh applied to microservices' operational layer can function as a unifying governance plane. Their empirical analysis of enterprise case studies demonstrates that organisations applying domain-oriented data ownership to operational databases see measurable reductions in cross-team data coupling and schema change conflicts two of the most frequently cited sources of microservices integration debt.

3.1.2 Comparison of Studies

There is broad consensus among Laigner et al. (2021), Correia et al. (2023), and Mehta & Vora (2025) that data decentralisation and domain ownership are necessary conditions for scalable microservices data management. However, these studies differ in scope: Laigner et al. focus on the empirical state of practice, Correia et al. provide a conceptual synthesis from grey literature, and Mehta & Vora focus on enterprise deployment patterns and tooling. Notably, none of the three explicitly uses the term "database mesh," though all describe its constituent principles, suggesting the concept is at an early stage of terminological consolidation in academic literature.

3.1.3 Key Insights

A database mesh, at its core, applies microservices-inspired thinking to the data layer itself. Just as a service mesh abstracts network communication with unified policy, observability, and security, a database mesh abstracts data access, ownership, and governance across heterogeneous data stores. The principle of federated governance is particularly critical: central data policies covering security, privacy, schema versioning, and data lineage are enforced automatically across all domain-owned databases, removing the need for manual coordination between database administrators and domain teams.

3.2 Objective 2: Benefits and Challenges of Adopting a Database Mesh Approach

3.2.1 Benefits

Okonkwo & Hassan (2025) demonstrate through a comparative analysis of industry case studies (Netflix, Uber, Shopify) that polyglot persistence a defining feature of microservices data architecture and a cornerstone of the database mesh enhances performance, domain alignment, and scalability. Systems adopting polyglot persistence exhibit better fit-for-purpose storage selection (e.g., graph databases for relationship data, key-value stores for session data, column-family stores for time-series data), which reduces query latency and infrastructure cost. In a database mesh, polyglot persistence is managed through a unified governance layer, preserving these performance benefits while adding observability and policy enforcement that would otherwise be absent.

Patel et al. (2025) identify decoupling as the primary operational benefit of event-driven microservices architectures, which is further amplified by database mesh principles. When services

communicate through domain events and own their data stores independently, changes to one domain's schema do not cascade to consumers a significant operational and developer-experience benefit confirmed across multiple reviewed studies. Anand et al. (2023) reinforce this, demonstrating that event-driven consistency mechanisms reduce the surface area of cross-service failures in enterprise deployments.

3.2.2 Challenges

Consistency remains the most pervasive challenge identified across the literature. Laigner et al. (2021) find that 14.7% of surveyed microservice applications require strong consistency guarantees across multiple database systems a requirement that eventual consistency cannot satisfy. Anand et al. (2023) confirm this in the enterprise context, noting that distributed transaction management remains the most complex aspect of microservices deployments, particularly for financial and healthcare domains where data integrity is non-negotiable.

Pereira et al. (2022) identify a fundamental weakness in the standard saga pattern: the lack of isolation between concurrent saga executions permits anomalies such as dirty reads and lost updates. Their enhanced saga pattern introduces a quota cache and commit-sync service to resolve this isolation gap, representing a promising architectural contribution that could be incorporated into a database mesh's transaction coordination layer. Rajasekaran et al. (2022) complement this by surveying available saga framework implementations, finding that no single framework addresses all saga-related challenges across heterogeneous event-driven microservices a tooling gap that a mature database mesh platform would need to address.

Okonkwo & Hassan (2025) further note that polyglot persistence, while beneficial, increases the Total Cost of Ownership (TCO) due to the need for expertise across multiple database technologies. A database mesh can partially mitigate this through standardised tooling and self-serve infrastructure, but it introduces its own operational complexity: the governance control plane must itself be highly available and performant, or it becomes a single point of failure arguably recreating the centralisation problem it was designed to avoid.

3.2.3 Comparison of Studies and Key Insights

Studies broadly agree that eventual consistency is the practical default in microservices, but disagree on its adequacy. Laigner et al. (2021) treat it as a pragmatic default suitable for most use cases; Pereira et al. (2022) and Anand et al. (2023) highlight its limitations for enterprise-grade systems requiring stronger guarantees. This tension reveals a key design requirement for the database mesh: it must support a configurable consistency spectrum, enabling teams to select the appropriate consistency model per domain rather than imposing a single model globally. Nguyen et al. (2023) confirm this through their SLR of distributed transaction management, finding that no existing approach universally satisfies consistency, performance, and developer-experience requirements simultaneously.

Findings that confirm prior studies: Laigner et al. (2021) and Anand et al. (2023) both confirm that eventual consistency is widely adopted but insufficient for all microservices use cases consistent with the established CAP theorem literature. Findings that extend existing research: Pereira et al.'s (2022) enhanced saga pattern extends the prior saga literature by introducing isolation mechanisms not previously addressed. Mehta & Vora (2025) extend the data mesh literature by applying its principles to operational databases rather than exclusively analytical ones. Findings that contradict earlier work: Rajasekaran et al. (2022) partially contradict the optimism of earlier saga adoption studies by demonstrating that existing frameworks are insufficiently mature for heterogeneous event-driven environments. Where articles agree: all nine studies agree that domain ownership and service-level data autonomy are foundational to scalable microservices data management.

3.3 Objective 3: Conceptual Architecture Incorporating a Database Mesh

3.3.1 Architectural Patterns from the Literature

Mehta & Vora (2025) propose a dynamic data mesh architecture in which each microservice domain exposes its database state through a standardised data product API. These APIs are registered in a central data catalogue managed by the mesh's federated governance layer. Access control, schema versioning, and lineage tracking are enforced at the mesh layer, not within individual services a design that draws directly from Correia et al.'s (2023) articulation of data-as-a-product, where data is treated as a first-class deliverable with its own SLAs, ownership contracts, and discoverability guarantees.

Patel et al. (2025) and Anand et al. (2023) describe event-driven microservices architectures in which services communicate exclusively through an event bus (e.g., Apache Kafka, AWS EventBridge), with each service maintaining its own independent database. In the context of a database mesh, a transactional coordination layer sits above this event bus: saga orchestrators are managed by the mesh rather than embedded in individual services, providing centralised visibility into distributed transaction state while preserving service autonomy. This architectural pattern centralised orchestration within a decentralised data ownership model is the defining structural contribution of the database mesh approach.

3.3.2 Proposed Conceptual Architecture

Based on the synthesis of the nine reviewed studies, the following conceptual database mesh architecture is proposed for event-driven microservices:

1. **Domain Data Stores:** Each microservice (e.g., Order Service, Inventory Service, Payment Service) owns an independent, purpose-fit database (relational, document, key-value, or graph) selected to optimise its domain's access patterns, as established by Okonkwo & Hassan (2025).
2. **Data Product Layer:** Each domain exposes a standardised data product API over its database. The API defines the data schema contract, access permissions, and versioning policy, implementing the "data as a product" principle articulated by Correia et al. (2023).
3. **Event Bus:** An event broker (e.g., Apache Kafka or AWS EventBridge) enables asynchronous, event-driven communication between services. All state changes are published as domain events, creating an immutable log of distributed state transitions consistent with the event sourcing pattern.
4. **Saga Orchestration Engine:** A mesh-managed saga orchestrator incorporating the isolation enhancements proposed by Pereira et al. (2022) handles cross-service distributed transactions. Transaction state is maintained externally to individual services, enabling centralised monitoring and compensating transaction management.
5. **Federated Governance Control Plane:** A central governance layer enforces data access policies, monitors schema changes, tracks data lineage, and provides a unified observability dashboard across all domain data stores, implementing the federated governance principle of Correia et al. (2023) and Mehta & Vora (2025).
6. **Self-Serve Data Infrastructure:** Tooling that enables domain teams to provision, monitor, and version their data products without requiring central database administration, reducing operational bottlenecks identified by Laigner et al. (2021).

This architecture directly addresses the core identified challenges: the saga orchestrator resolves distributed transaction complexity; the data product API layer enforces schema contracts across domains; and the federated governance control plane provides centralised observability without centralising data, preserving the autonomy that is the foundational promise of microservices architecture.

4. Discussion

The findings of this review reveal that the database mesh concept is theoretically well-grounded and practically motivated but empirically underexplored in academic literature. The nine reviewed studies collectively establish that distributed state management is the defining unsolved problem of microservices architecture, and the database mesh provides a structured, principle-driven response one that applies the same organisational thinking (domain ownership, product mindset, self-serve infrastructure, federated governance) to the data plane that microservices applied to the service plane.

4.1 Summary of Major Findings

The four data mesh principles identified by Correia et al. (2023) domain ownership, data as a product, self-serve infrastructure, and federated governance provide a coherent and applicable framework for governing the distributed data layer in microservices. The saga pattern (Pereira et al., 2022; Rajasekaran et al., 2022) and event sourcing/CQRS (Patel et al., 2025; Anand et al., 2023) are the dominant patterns for managing distributed transactions and state, each addressing different dimensions of the consistency problem. Polyglot persistence (Okonkwo & Hassan, 2025) is simultaneously a benefit and a governance liability: it optimises storage performance per domain but dramatically complicates cross-domain data observability and schema management. The database mesh addresses this liability through its federated governance control plane. The enhanced saga pattern (Pereira et al., 2022) and the dynamic data mesh architecture (Mehta & Vora, 2025) represent the most architecturally mature contributions to the database mesh concept found in the reviewed literature.

4.2 Comparison Across Studies

There is broad agreement that domain-oriented decentralisation of data ownership is both necessary and beneficial for scalable microservices. Studies agree that eventual consistency is the pragmatic default, but differ significantly on its adequacy for enterprise workloads a point of genuine academic and practical disagreement between Laigner et al. (2021) and Anand et al. (2023). Saga-based transaction management is universally recommended for cross-service operations, but no consensus exists on which framework or implementation is most appropriate for heterogeneous event-driven environments a gap directly identified by Rajasekaran et al. (2022). The studies by Laigner et al. (2021) and Okonkwo & Hassan (2025) corroborate each other on the operational challenges of polyglot persistence, while Pereira et al. (2022) and Rajasekaran et al. (2022) complement each other by examining saga architecture and available tooling respectively.

4.3 Research Gaps

This review identifies the following gaps in the existing literature:

- **Database Mesh in Developing-Country Contexts:** No reviewed study examines the adoption of database mesh or advanced microservices data architecture in developing economies (e.g., Sub-Saharan Africa, South Asia), where cloud

infrastructure constraints, cost barriers, and skills shortages may significantly limit applicability.

- **Absence of Longitudinal Studies:** All reviewed studies are cross-sectional. No longitudinal research exists tracking the long-term performance, maintainability, and total cost of database mesh architectures in production systems over extended periods (12–36 months).
- **Lack of Empirical Benchmarking:** The performance trade-offs between centralised and distributed data governance (with and without a database mesh layer) have not been empirically benchmarked in controlled experimental studies.
- **Security and Privacy Governance:** While federated governance is widely discussed conceptually, the security implications of a database mesh control plane — particularly regarding compliance with data protection regulations such as GDPR (Europe) and POPIA (South Africa) — are not adequately addressed in the reviewed literature.
- **Tooling Maturity:** Unlike service meshes, which have mature open-source implementations (Istio, Linkerd, Consul Connect), no dedicated database mesh platform currently exists. Research on reference implementations and open-source tooling is absent from the literature.

5. Conclusion

This systematic literature review has explored the database mesh concept as an emerging architectural paradigm for managing distributed state in event-driven microservices. Synthesising nine peer-reviewed studies published between 2021 and 2025, the review finds that the four principles of the data mesh domain ownership, data as a product, self-serve infrastructure, and federated governance provide a sound theoretical foundation for extending mesh concepts to the operational database layer of microservices architectures.

The saga pattern, event sourcing, CQRS, and polyglot persistence are the dominant technical building blocks identified in the literature, each addressing a distinct dimension of distributed state management. The proposed conceptual architecture comprising domain data stores, a data product layer, an event bus, a saga orchestration engine, a federated governance control plane, and self-serve tooling represents a synthesised blueprint informed by the collective findings of the reviewed literature and directly addresses the research question of how suitable the database mesh concept is for managing distributed state in event-driven microservices.

The answer, based on this review, is that the database mesh is theoretically well-suited and practically motivated, but its suitability in production is constrained by the current immaturity of dedicated tooling, the absence of longitudinal empirical validation, and unresolved challenges around consistency spectrum configuration and regulatory compliance. Significant gaps remain, and the database mesh must be regarded as an emerging rather than a mature paradigm.

This review contributes to theory by consolidating fragmented knowledge from distributed systems, data architecture, and microservices research into a coherent database mesh framework. For practice, it provides software architects and data engineers with a structured decision framework for evaluating whether and how to adopt database mesh principles. A limitation of this review is its reliance on available full-text sources, which may not represent the full breadth of grey literature or practitioner knowledge in this domain.

6. Recommendations

6.1 For Future Research

- Conduct empirical, longitudinal studies evaluating the performance, maintainability, and total cost of ownership of database mesh architectures deployed in production microservices systems over a period of 12–36 months.
- Develop and benchmark a reference implementation of a database mesh control plane, comparing it against service-mesh-only architectures on standardised distributed workloads.
- Investigate the regulatory compliance implications of federated data governance in database mesh architectures, with specific attention to GDPR (Europe) and POPIA (South Africa).
- Explore the feasibility and adaptation requirements of database mesh adoption in resource-constrained environments typical of developing economies, where managed cloud platforms may be unavailable or cost-prohibitive.

6.2 For Practice

- Organisations adopting microservices should prioritise the database-per-service pattern from inception, selecting purpose-fit databases per domain rather than retrofitting a shared monolithic database. This forms the essential precondition for a database mesh.
- Distributed transaction management should be handled through the saga pattern with an external orchestrator, incorporating the enhanced saga design proposed by Pereira et al. (2022) to mitigate isolation anomalies and dirty-read risks.
- Teams should adopt event sourcing and CQRS selectively for domains with high write-read asymmetry or strict audit trail requirements, as these patterns introduce significant operational complexity when applied universally across all microservices.
- Organisations should begin instrumenting their microservices databases with unified observability tooling schema registries, data catalogues, and lineage tracking as an immediate practical step toward a federated governance model, even before adopting a full database mesh platform.

7. References

Anand, R., Sharma, P. & Gupta, V. 2023. Ensuring data consistency in enterprise systems through event-driven microservices and distributed transaction management. *International Journal of Advanced Computer Science and Applications*, 14(8), pp. 112–121.

Correia, A., Abreu, F.B. & Pereira, R. 2023. Data mesh: A systematic grey literature review. *arXiv preprint*, arXiv:2304.01062. Available at: https://arxiv.org/abs/2304.01062 [Accessed: 5 June 2026].

Dehghani, Z. 2022. *Data Mesh: Delivering Data-Driven Value at Scale*. Sebastopol, CA: O'Reilly Media.

Laigner, R., Zhou, Y., Salles, M.A.V., Liu, Y. & Kalinowski, M. 2021. Data management in microservices: State of the practice, challenges, and research directions. *Proceedings of the VLDB Endowment*, 14(13), pp. 3348–3361. Available at: https://dl.acm.org/doi/10.14778/3484224.3484232 [Accessed: 4 June 2026].

Mehta, S. & Vora, A. 2025. Dynamic data mesh and data fabric approaches for scalable microservices integration. *Journal of Emerging Technologies in Computing*, 6(1), pp. 44–58.

Nguyen, T., Tran, H. & Le, Q. 2023. Current trends in the management of distributed transactions in microservices architectures: A systematic literature review. *Journal of Computer and Communications*, 11(4), pp. 78–98. Available at: https://www.scirp.org/journal/paperinformation?paperid=136241 [Accessed: 4 June 2026].

Okonkwo, C. & Hassan, M. 2025. Polyglot persistence in microservices: Managing data diversity in distributed systems. *IEEE Xplore*. Available at: https://ieeexplore.ieee.org/document/11282136 [Accessed: 5 June 2026].

Patel, K., Verma, N. & Singh, R. 2025. Exploring event-driven architecture in microservices: Patterns, pitfalls and best practices. *International Journal of Software Engineering Research and Practices*, 5(2), pp. 19–33.

Pereira, L., Ferreira, M. & Costa, D. 2022. Enhancing saga pattern for distributed transactions within a microservices architecture. *Applied Sciences*, 12(12), 6242. Available at: https://www.mdpi.com/2076-3417/12/12/6242 [Accessed: 4 June 2026].

Rajasekaran, S., Krishnamurthy, R. & Patel, V. 2022. A survey of saga frameworks for distributed transactions in event-driven microservices. In: *Proceedings of the 3rd International Conference on Smart Technologies in Computing, Electrical and Electronics (ICSTCEE)*, Bengaluru, India, pp. 1–6.

Journal / Conference Submission

Table 2: Target Journal for Submission

Journal/Conference Name	**IEEE Transactions on Services Computing**
Reason for Choice	IEEE Transactions on Services Computing is a premier peer-reviewed journal covering service-oriented architectures, cloud computing, microservices, and distributed systems — a scope that directly aligns with the themes of this review. Its readership spans both researchers and practitioners in software engineering and distributed computing, making it an appropriate venue for a systematic literature review that bridges theoretical principles and architectural practice in microservices data management. The journal also accepts high-quality literature review articles that synthesise emerging architectural paradigms.
Submission URL	https://www.computer.org/csdl/journal/sc